

RL Arm Motion Project

Physics Constraints & Ablation Study

Session Report — March 2026

Author: Ranjot Sandhu · AI Partner: Claude

This report documents the physics-based training constraints added to the robotic arm reinforcement learning environment, the reasoning behind each constraint and its equations, a Q&A log of the session discussion, and the results of an ablation study that quantifies how each constraint individually affects arm motion.

Table of Contents

- 1. Project Background & Branch Strategy
- 2. Physics Constraints Added to the Worktree Branch
 - 2.1 A — Gravity Torque
 - 2.2 C — Joint Acceleration Limits
 - 2.3 J — Energy Budget Tracking
 - 2.4 K — Jerk Penalty
- 3. Session Q&A; Log
- 4. Ablation Study
 - 4.1 Methodology
 - 4.2 Results Table
 - 4.3 Interpretation of Each Constraint
- 5. Summary & Recommendations

1. Project Background & Branch Strategy

The **RL Arm Motion** project trains a 2-DOF planar robotic arm using Proximal Policy Optimisation (PPO) in a Gymnasium-compatible environment. The arm starts vertically downward and must reach a target position (EAST / WEST / NORTH / HEIGHT) and hold it.

Two branches are maintained:

- **main** — stable, optimised baseline code (code efficiency improvements, PPO hyperparameter tuning, GUI speed slider, goal-tolerance fixes).
- **claude/trusting-chaum** (worktree) — physics-constraints feature branch. Contains all physics code described in this document. Not yet merged into main; will be merged via Pull Request after validation.

2. Physics Constraints Added to the Worktree Branch

Four physics-based constraints were implemented in `src/rl_armMotion/two_d/environments/task_env.py`. Each is labelled by the letter used in the code comments.

2.1 A — Gravity Torque

Without gravity torque, the RL agent treats moving the arm horizontally the same as moving it vertically. In reality, holding a horizontal arm costs significantly more energy than hanging straight down, because gravity continuously pulls the links downward. Adding this term forces the agent to learn energy-efficient paths — it naturally discovers that swinging through lower positions is cheaper than fighting gravity directly.

Equation (standard planar serial-chain formula):

$$\tau_{i} = -g * \sum_{k \geq i} m_k * (x_{com_k} - x_{joint_i})$$

where $g = 9.81 \text{ m/s}^2$, m_k is the mass of link k , x_{com_k} is its centre-of-mass x-coordinate, and x_{joint_i} is the x-coordinate of joint i . Source: standard Newton-Euler formulation (Spong, Hutchinson & Vidyasagar — *Robot Modeling and Control*).

Numbers used:

- $g = 9.81 \text{ m/s}^2$ — standard gravitational acceleration
- `link_lengths = [1.0, 0.8] m`, `masses = [2.0, 1.5] kg` — from your 2DOF preset in `arm_config.py`
- `gravity_penalty_coeff = 0.003` — tuned so the gravity penalty is ~ 0.09 reward units at full horizontal extension, keeping it smaller than the primary distance term ($-2.0 \times \text{distance}$).

2.2 C — Joint Acceleration Limits

Without this, the agent can command the arm to jump from full speed in one direction to full speed in the opposite direction in a single 10 ms timestep. That is physically impossible for a real motor. The limit forces the agent to plan ahead and decelerate before reversing — producing smooth arc trajectories rather than jerky back-and-forth motion.

Implementation:

```
max_delta_vel = max_joint_accel * dt = 8.0 * 0.01 = 0.08 rad/s per step
delta_vel = clip(vel_cmd - prev_velocities, -max_delta_vel, max_delta_vel)
```

Numbers used:

- $\text{max_joint_accel} = 8.0 \text{ rad/s}^2$ — chosen so the arm reaches full speed ($\text{velocity_limit} = 2.0 \text{ rad/s}$) in 0.25 seconds (25 steps). Realistic for a lightweight 2-DOF arm; industrial arms typically have $5\text{--}15 \text{ rad/s}^2$.
- $\text{accel_penalty_coeff} = 0.05$ — maximum acceleration on both joints costs ~ 0.05 reward units.

2.3 J — Energy Budget Tracking

Tracks total mechanical energy spent per episode. It does not cut the episode off but penalises wasted energy in the reward. This nudges the agent toward efficient paths — a direct arc costs less than oscillating back and forth even if both eventually reach the goal.

Equation (mechanical power formula):

```
step_energy = |tau_gravity| . |omega| . dt [Joules]
```

$P = \tau \cdot \omega$ (Power = Torque $\times$ angular velocity), $\text{Energy} = P \times dt$. This is basic physics (Joules = $\text{N}\cdot\text{m} \times \text{rad/s} \times \text{s}$). No hard episode cutoff — tracked in the info dictionary and penalised softly in the reward.

2.4 K — Jerk Penalty

Jerk is the rate of change of acceleration — the standard smoothness metric in robotics trajectory planning. Penalising jerk encourages the arm to follow smooth, arc-like trajectories rather than abrupt start-stop motion, which is important for both realism and mechanical longevity.

Equation:

```
jerk = delta(delta_vel) / dt
jerk_norm = clip( mean|jerk| / (max_accel / dt), 0, 1 )
```

Source: Flash & Hogan (1985), *The coordination of arm movements: an experimentally confirmed mathematical model* — the foundational paper on minimum-jerk trajectories in human and robotic arm motion.

Numbers used:

- $\text{jerk_penalty_coeff} = 0.02$ — small because jerk is naturally high during start/stop, which is unavoidable. Coefficient kept low so it shapes trajectories without dominating the primary goal-reaching signal.

3. Session Q&A; Log

The following questions were asked during this session. Each answer is reproduced in full.

Q: Why do you want to add Gravity Torque, Joint Acceleration Limits, and Energy Budget to the arm model?

Gravity Torque: Without it the RL agent treats horizontal and vertical motion identically. Real motors must work harder to hold a horizontal arm against gravity. Adding the torque term teaches the agent energy-efficient paths — it naturally finds that swinging low is cheaper than fighting gravity directly.

Joint Acceleration Limits: Without limits, the agent can reverse direction instantly in one timestep — physically impossible for real motors. The limit forces the agent to plan ahead, decelerate before reversing, and produce smooth arc trajectories.

Energy Budget: Tracks mechanical work done against gravity each step. Soft reward penalty nudges the agent toward efficient paths — a direct arc costs less energy than oscillating back and forth.

Q: Where did the numbers in the equations come from?

$g = 9.81 \text{ m/s}^2$ — standard gravitational constant.

link_lengths and masses — taken directly from your existing 2DOF preset configuration in `arm_config.py` ([1.0, 0.8] m, [2.0, 1.5] kg).

$\text{max_joint_accel} = 8.0 \text{ rad/s}^2$ — chosen so the arm reaches its full velocity limit (2.0 rad/s) in 0.25 seconds, which is realistic for lightweight robotic arms (industrial standard: 5–15 rad/s²).

Penalty coefficients (0.003, 0.05, 0.02) — manually tuned so each physics penalty stays smaller than the primary distance penalty ($-2.0 \times \text{goal_distance}$) and does not dominate the reward signal.

Gravity torque formula — standard Newton-Euler serial chain formula from Spong, Hutchinson & Vidyasagar *Robot Modeling and Control*. **Jerk penalty** — based on Flash & Hogan (1985) minimum-jerk trajectory research.

Q: Did we quantify how each constraint changes the learning process individually?

Not initially — all three constraints were added in a single commit without ablation testing. This was identified as a gap. An ablation study was then run (see Section 4) to measure the individual effect of each constraint on success rate, final distance to goal, motion smoothness, jerk, and total energy per episode.

Q: Can you just run the ablation study here instead of writing code to a file?

Yes — the study was run directly in the terminal using the project's virtual environment. A heuristic P-controller (proportional gain = 3.0) was used as the policy so results are not confounded by RL training variance. 60 episodes were run per configuration. Results are presented in Section 4.

Q: Why add a jerk penalty specifically? Is it in the physics commit?

The jerk penalty was added to the worktree branch (claude/trusting-chaum) alongside the other three physics constraints. It was added because jerk (rate of change of acceleration) is the robotics standard for trajectory smoothness. Minimum-jerk trajectories (Flash & Hogan 1985) are the closest mathematical model to how real human arms move. Penalising jerk during training produces policies that generate smooth, arc-like motion rather than abrupt start-stop commands.

4. Ablation Study

4.1 Methodology

A heuristic P-controller (proportional gain 3.0, with damping near the goal) was used as the policy so results are not confounded by RL training stochasticity. Each of the six physics configurations was evaluated over **60 episodes** from the standard initial state (arm vertical downward). The task is the HEIGHT goal — end-effector must reach shoulder height ($y = 0$) and hold for 20 consecutive steps.

Metrics recorded:

- **Success %** — percentage of episodes where the arm held the goal region for 20 consecutive steps.
- **Avg Dist (m)** — mean distance from end-effector to goal at episode end.
- **Smoothness** ↓ — mean normalised $|\Delta \text{velocity}|$ per step. Lower = smoother motion.
- **Jerk** ↓ — mean normalised rate of acceleration change per step. Lower = smoother.
- **Energy** ↓ — mean total mechanical energy $|\text{torque} \times \text{velocity}| \times dt$ per episode (J). Lower = more efficient.

4.2 Results Table

Configuration	Success%	Dist (m)	Smooth ↓	Jerk ↓	Energy (J) ↓
Baseline (no physics)	100.0%	0.0788	0.1227	0.0015	29.361
+Gravity penalty only	100.0%	0.0788	0.1227	0.0015	29.361
+Accel limit only	100.0%	0.0793	0.1175	0.0001	29.328
+Energy penalty only	100.0%	0.0788	0.1227	0.0015	29.361
+Gravity + Accel	100.0%	0.0793	0.1175	0.0001	29.328
+All (Gravity+Accel+Energy+Jerk)	100.0%	0.0793	0.1175	0.0001	29.328

Highlighted rows (+Accel limit only, +Gravity + Accel, +All) show the configurations where the acceleration limit is active — the only rows with measurably different smoothness and jerk values.

4.3 Interpretation of Each Constraint

Gravity Penalty — No measurable standalone effect on motion

The gravity penalty reduces the reward for high-effort arm configurations but does not change how the arm physically moves. With a heuristic controller (which ignores rewards), there is zero change in smoothness, jerk, or energy. During RL training the penalty matters — it discourages the trained policy from loitering in high-effort configurations — but it is a reward-shaping signal only, not a physical constraint.

Acceleration Limit — Only constraint that physically changes motion

Smoothness improved by 4.3% ($0.1227 \rightarrow 0.1175$) and jerk dropped by 93% ($0.0015 \rightarrow 0.0001$). The arm can no longer snap instantly to full speed — it ramps up and down, producing visibly smoother motion. The tiny distance increase ($+0.0005$ m) is because the arm takes slightly longer to decelerate into the goal region. This is the most impactful physical constraint.

Energy Penalty — No measurable standalone effect on motion

Like the gravity penalty, this is a reward-shaping signal. It penalises mechanical work done against gravity per step, nudging a trained RL policy toward efficient paths. With a heuristic controller it produces zero change in the metrics. Valuable for RL training; not a hard physical constraint.

All Combined — Same as Accel Limit alone (for heuristic policy)

When all four constraints are active, the results are identical to the accel-limit-only configuration. This confirms that gravity, energy, and jerk penalties are reward-shaping signals that only matter when an RL agent is actively optimising the reward. The acceleration limit is the single constraint that directly enforces physical realism in every episode regardless of the policy.

5. Summary & Recommendations

1. **Acceleration limit is the only constraint that physically enforces smooth motion.** It reduced jerk by 93% and smoothness error by 4.3% independently of the policy being used.
2. **Gravity torque and energy budget are RL training signals.** They shape the reward landscape to teach the agent efficient, energy-aware behaviour, but do not constrain the raw physics of motion.
3. **Jerk penalty complements the acceleration limit.** It adds a softer, graduated cost on abrupt acceleration changes that encourages the smoothest possible trajectories within the hard acceleration bound.
4. **All constraints together are recommended** for production training. The acceleration limit gives physical realism; gravity + energy + jerk give the RL agent meaningful signals to learn efficient, smooth, realistic motion.

Next Steps

- Merge **claude/trusting-chaum** into main via Pull Request once validated.
- Retrain models using the full physics environment (recommended: 500k–1M timesteps).
- Run a full RL ablation study (not heuristic) to measure how each penalty affects policy convergence speed and final reward.
- Consider adding the **outer-arc trajectory corridor** as a curriculum waypoint rather than a hard constraint, to guide the arm toward efficient arcing paths during early training.
- Evaluate using SAC (off-policy) in addition to PPO — SAC typically converges faster on continuous control tasks with shaped rewards.